Excellent observation — and this is **totally normal** given what your engine is doing right now. Let's break down **why it feels slow**, what's happening behind the scenes, and how to speed it up without breaking anything.

🧠 Why it's taking so long

When you run:

```
python matcher.py
```

and see:

```
🚀 Starting Two-Way Hybrid Matching Engine...
🔍 Matching Startups → Investors ...
```

you're triggering a **massive multi-step pipeline**. Let's walk through exactly what it's doing:

🧩 Step 1. Data loading

```
startups = load_startups_from_csv()
investors = load_investors_from_csv()
```

☑ Reads **both CSV files fully** into memory. If your CSVs have hundreds or thousands of records, that's already a few seconds.

🧩 Step 2. Elasticsearch index recreation

```
create_index(INVESTOR_INDEX)
index_investors(investors)
```

By design, `create_index()` deletes any old index and **re-creates** it. Then `index_investors()`:

- Computes an **embedding** for every investor description using `SentenceTransformer("all-MiniLM-L6-v2")`
- Uploads each one to Elasticsearch via `es.index(...)`

This means:

> One HTTP request per investor, **plus** an embedding computation (matrix math) for each.

If you have 1000+ investors, this step alone can take **1–5 minutes** depending on:

- Model load time (first use of MiniLM is slow — it downloads & warms up)
- Network speed (to Elastic Cloud)
- Elasticsearch indexing latency (each index call ≈ 50–200 ms)

🧩 Step 3. Matching loop

```
for startup in startups:
    startup_emb = compute_embedding(startup.description)
    es_hits = search_similar_investors(startup_emb, top_k)
```

For every startup:

- It computes another **embedding** (slow matrix math on CPU)
- Runs a **KNN vector search** on Elasticsearch
- Applies your **rule-based scoring** logic
- Optionally calls the **Groq LLM API** for justification

So for N startups × M investors, you're easily doing **hundreds of API and compute calls**.

🧩 Step 4. Groq Justifications

If you enabled:

```
GROQ_ENABLED=true
```

then every high-scoring pair triggers:

```
get_llm_justification(...)
```

That's a **network call to the Groq API**, which can take **2–5 seconds per justification**. Multiply that by every match above threshold — it explodes quickly.

🚦 Typical runtime (rough estimate)

Dataset size	Approx runtime (CPU, local Elastic Cloud)
10 startups × 20 investors	~30–60 s
100 × 200	3–8 min
1000 × 1000	20 min +

If Groq is enabled, it can double.

⚡ How to make it faster

☑ 1. Cache embeddings

Add a small caching layer:

```
from functools import lru_cache

@lru_cache(maxsize=None)
def compute_embedding_cached(text):
    return compute_embedding(text)
```

Then replace all `compute_embedding()` calls with `compute_embedding_cached()`. This avoids recomputing the same text repeatedly.

☑ 2. Don't recreate the index every run

Replace:

```
create_index(INVESTOR_INDEX)
index_investors(investors)
```

with:

```
if not es.indices.exists(index=INVESTOR_INDEX):
    create_index(INVESTOR_INDEX)
    index_investors(investors)
```

This way, the embeddings are reused instead of rebuilt each time.

☑ 3. Reduce data temporarily

While testing, use small CSVs:

```
head -n 30 investors.csv > investors_small.csv
head -n 30 startups.csv > startups_small.csv
```

Then point to them:

```
load_investors_from_csv("investors_small.csv")
load_startups_from_csv("startups_small.csv")
```

☑ 4. Disable LLM justifications during testing

In `.env`:

```
GROQ_ENABLED=false
```

This will skip the slow Groq API calls until you're ready for final runs.

☑ 5. Use `top_k=5` instead of 30

Every search asks for `top_k` matches from Elasticsearch. Each KNN search is O(log N) but high K adds cost:

```
matches = two_way_match(top_k=5)
```

🧩 Optional: use batch indexing

Right now you call:

```
es.index(...)
```

for each investor/startup.

You can speed that up with Elasticsearch's **bulk API**, batching 100–500 docs per request.

☑ TL;DR

It's slow because it's:

- Re-creating and re-indexing everything every time
- Computing embeddings on CPU for each record
- Possibly generating Groq justifications (network)
- Running 2 full passes (startups → investors and investors → startups)

Quick wins

1. Comment out Groq (`GROQ_ENABLED=false`)
2. Skip index recreation on each run

3. Use small CSV subsets while testing
4. Cache embeddings

Would you like me to show a small **optimized version** of your indexing + matching loop (with caching, bulk indexing, and progress bars using `tqdm`)? That'll make it roughly **3–5× faster** for large data while showing you live progress.